CourseFit — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── samples/
│   ├── background.txt
│   └── course.txt
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── background_store.py
│   │   └── retriever.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── extractor.py
│   ├── matcher.py
│   ├── pipeline.py
│   └── schemas.py
├── tests/
│   ├── integration/
│   │   └── test_live_check.py
│   └── unit/
│       ├── test_config.py
│       └── test_matcher.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
```

```
├── README.md
└── requirements.txt
```

Setup files

`requirements.txt`:

```
chromadb
langchain-community
langchain-openai
langchain-text-splitters
openai
pydantic
python-dotenv
```

`.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. `LICENSE` is the MIT text. Every folder under `src/` gets an empty `__init__.py`.

Sample background and course (`samples/`)

```
# samples/background.txt
Completed Calculus I and II with A grades.
Took MATH 221: Matrices and Vectors (linear algebra).
Two years of Python programming for data analysis.
```

```
# samples/course.txt
Advanced Machine Learning.
Prerequisites: linear algebra, Python, and intro statistics.
```

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str
    match_cutoff: float


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        match_cutoff=float(os.getenv("MATCH_CUTOFF", "0.8")),
    )
```

```
# src/utils/logger.py
import logging
import os


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
```

```
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"


def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

The typed shapes and the engine

```
# src/schemas.py
from typing import List

from pydantic import BaseModel


class RequiredPrereqs(BaseModel):
    prereqs: List[str] = []


class ReadinessReport(BaseModel):
    score: int
    met: List[str] = []
    missing: List[str] = []
```

```
# src/matcher.py
# For each prerequisite, ask a distance() function how far the closest
# background chunk is; at or below the cutoff counts as met. distance() is
# passed in, so the decision is testable with no model or embeddings.
from src.schemas import ReadinessReport

DEFAULT_CUTOFF = 0.8


def check_prereqs(required, distance, cutoff: float = DEFAULT_CUTOFF):
```

```
    met = []
    missing = []
    for prereq in required:
        if distance(prereq) <= cutoff:
            met.append(prereq)
        else:
            missing.append(prereq)
    total = len(required)
    if total == 0:
        score = 100
    else:
        score = round(100 * len(met) / total)
    return ReadinessReport(score=score, met=met, missing=missing)
```

The RAG layer and the extractor

```
# src/rag/background_store.py
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

import chromadb

COLLECTION_NAME = "background"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")


def build_background_store(background_text: str):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=300, chunk_overlap=40,
    )
    chunks = splitter.split_text(background_text)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    for index in range(len(chunks)):
        ids.append(f"chunk-{index}")
    collection.add(
        ids=ids, documents=chunks,
        embeddings=EMBEDDINGS.embed_documents(chunks),
```

```
    )
    return collection
```

```
# src/rag/retriever.py
from src.rag.background_store import EMBEDDINGS

NO_MATCH_DISTANCE = 2.0


def best_distance(collection, prereq: str) -> float:
    if collection.count() == 0:
        return NO_MATCH_DISTANCE
    results = collection.query(
        query_embeddings=[EMBEDDINGS.embed_query(prereq)], n_results=1,
    )
    distances = results["distances"][0]
    if not distances:
        return NO_MATCH_DISTANCE
    return distances[0]
```

```
# src/extractor.py
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import RequiredPrereqs

PREREQS_PROMPT = (
    "From the course description below, list the prerequisites as short "
    "names (e.g. linear algebra, Python, statistics). Return an empty list "
    "if none are stated.\n\nCourse:\n{course}"
)


def extract_required_prereqs(model, course_text: str) -> RequiredPrereqs:
    prompt = ChatPromptTemplate.from_template(PREREQS_PROMPT)
    chain = prompt | model.with_structured_output(RequiredPrereqs)
    return chain.invoke({"course": course_text})
```

The pipeline

```
# src/pipeline.py
from src.extractor import extract_required_prereqs
from src.matcher import check_prereqs
from src.rag.background_store import build_background_store
```

```
from src.rag.retriever import best_distance
from src.schemas import ReadinessReport
from src.utils.logger import get_logger


def run_check(model, background_text: str, course_text: str,
              cutoff: float = 0.8) -> ReadinessReport:
    logger = get_logger(__name__)
    collection = build_background_store(background_text)
    required = extract_required_prereqs(model, course_text).prereqs

    def distance(prereq: str) -> float:
        return best_distance(collection, prereq)

    report = check_prereqs(required, distance, cutoff=cutoff)
    logger.info(
        f"met {len(report.met)}/{len(required)} prereqs "
        f"(readiness {report.score})"
    )
    return report
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_check


def main():
    if len(sys.argv) < 3:
        print("usage: python main.py <background-file> <course-file>")
        return
    with open(sys.argv[1]) as background_file:
        background_text = background_file.read()
    with open(sys.argv[2]) as course_file:
        course_text = course_file.read()

    config = load_config()
    model = get_model(config)
    report = run_check(model, background_text, course_text, config.match_cutoff)
    total = len(report.met) + len(report.missing)
```

```
    print(f"\nCourseFit readiness: {report.score}%  "
          f"({len(report.met)} of {total} prerequisites met)")
    print("\nMet:")
    for prereq in report.met:
        print(f"  - {prereq}")
    print("\nMissing:")
    for prereq in report.missing:
        print(f"  - {prereq}")


if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "COURSEFIT_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")


if __name__ == "__main__":
```

```
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_matcher.py
# Run from the repo root: python -m tests.unit.test_matcher
from src.matcher import check_prereqs

COVERED = {"linear algebra": 0.2, "python": 0.1}


def fake_distance(prereq: str) -> float:
    return COVERED.get(prereq.lower(), 1.5)


def test_met_and_missing() -> None:
    required = ["Linear algebra", "Python", "Statistics"]
    report = check_prereqs(required, fake_distance, cutoff=0.8)
    result = "OK"
    if report.missing != ["Statistics"]:
        result = "WRONG"
    if len(report.met) != 2:
        result = "WRONG"
    print(f"case 1 (2 met, 1 missing)  -> {result}")


def test_score_is_a_percentage() -> None:
    required = ["Linear algebra", "Python", "Statistics"]
    report = check_prereqs(required, fake_distance, cutoff=0.8)
    result = "OK"
    if report.score != 67:
        result = "WRONG"
    print(f"case 2 (score = 67)        -> {result}")


def test_no_prereqs_is_ready() -> None:
    report = check_prereqs([], fake_distance)
    result = "OK"
    if report.score != 100:
        result = "WRONG"
    print(f"case 3 (no prereqs -> 100) -> {result}")


if __name__ == "__main__":
    test_met_and_missing()
    test_score_is_a_percentage()
    test_no_prereqs_is_ready()
```

```
# tests/integration/test_live_check.py
# Real model + embeddings. Run from the repo root:
#   python -m tests.integration.test_live_check
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_check

BACKGROUND = (
    "Took MATH 221: Matrices and Vectors. Two years of Python for data work."
)
COURSE = "Prerequisites: linear algebra, Python, and intro statistics."


def test_meets_most_and_finds_gap() -> None:
    config = load_config()
    model = get_model(config)
    report = run_check(model, BACKGROUND, COURSE, config.match_cutoff)
    ok = report.score >= 50 and "Statistics" in report.missing
    print(f"case 1 (meets most, stats gap) -> {'OK' if ok else 'WRONG'}")


if __name__ == "__main__":
    test_meets_most_and_finds_gap()
```

Expected unit output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (2 met, 1 missing)  -> OK
case 2 (score = 67)        -> OK
case 3 (no prereqs -> 100) -> OK
```

The whole project, in one sentence

The course's prerequisites extracted as a list, the student's background made searchable by meaning, and each prerequisite matched by a distance the deterministic engine turns into a readiness score and a gap list — the same patterns as JobMatch, on an entirely different requirements-check.